

Abstract—This note documents how the open-source Python implementation of the Convex-Optimal Water-Power Flow (C-OWPF) derives each solve mode directly from the manuscript’s formulations, and maps every mode to the source that implements it. No new physics is introduced: the code solves the same successive linear approximation of the nonlinear water hydraulics and the LinDistFlow power model described in the paper.

I. THE PROBLEM SOLVED AT EACH STEP

Every mode reduces to a sequence of mixed-integer convex programs of the paper’s problem (P2). A single step minimizes the priced pump energy together with the priced network loss, subject to the linearized WDN hydraulics, the LinDistFlow PDN model, PV inverter capability, and voltage limits:

$$\min \sum_t \lambda_t^{\text{WDN}} \left(\underbrace{\sum_{ij \in \mathcal{P}} \hat{C}_{ij,t}^{\text{pump}}}_{\text{pump energy}} + \underbrace{\hat{C}_t^{\text{loss}}}_{\text{loss}} \right)$$

s.t. mass balance, junction/tank/reservoir heads;

lin. pipe head loss and (F/V)SP power;

$$\mathbf{v}^2 = \mathbf{R}(-\mathbf{p}) + \mathbf{X}(-\mathbf{q}) + \mathbf{v}_k; \quad v_{\min} \leq v_{n,t} \leq v_{\max}.$$

The pump power enters the PDN as a bus load; its reactive part is coupled through the power factor, $q^{\text{pump}} = p^{\text{pump}} \tan(\arccos \text{PF})$. Voltage-dependent shunt capacitors are folded into $\mathbf{R}, \mathbf{X}, \mathbf{v}_k$ exactly as in the paper’s Kekatos correction.

II. WATER (DECOUPLED) SOLVE MODES

Four modes share the linearized hydraulics and differ only in how the operating point and pump schedule are obtained.

- **EPANET** — rule-based baseline; pumps follow tank-level rules and EPANET reports the true nonlinear cost. No optimization; the cost reference (Scenario A).
- **direct** — one linearized MILP solved once about a nominal point, no relinearization. Fast on radial networks.
- **warmstart** — an EPANET-imposed schedule gives consistent flows/heads; hydraulics are linearized *once* about that point and solved. Robust on looped networks where *direct* fails.
- **optimize** — the full successive-linearization loop (Algorithm 1) wrapped in the voltage-aware schedule search below; realizes the reported savings.

III. COUPLED MODE

The coupled mode runs the successive-linearization loop of Fig. 1 and then a schedule search that, unlike the water-only search, scores every candidate on the *coupled* solve, so a schedule that is cheap for water yet causes an undervoltage on the feeder is rejected.

Schedule search is cost- and voltage-aware: each candidate is scored on the true total cost (pump + loss) and voltage feasibility.

Warm-started trust-region MILP: free binaries, warm-started at the incumbent’s linearisation, with a Hamming-distance cap of K flips — the MATLAB-style free-binary MILP made reliable (the trust region blocks the far-away schedule flips a cold MILP would take on a stale linearisation).

Concretely the search runs in four stages, ranking *coupled-feasible first* (water slack and voltage violation within tolerance), then lowest true cost: (1) EPANET baseline; (2) price-quantile candidates — pump only in the cheapest hours (plus a duty-preserving load shift on large networks); (3) warm-started trust-region MILP, $K = \max(2, \lfloor n_{\text{pump}} T/4 \rfloor)$; (4) 1-opt polish (single pump-hour flips, greediest-price first).

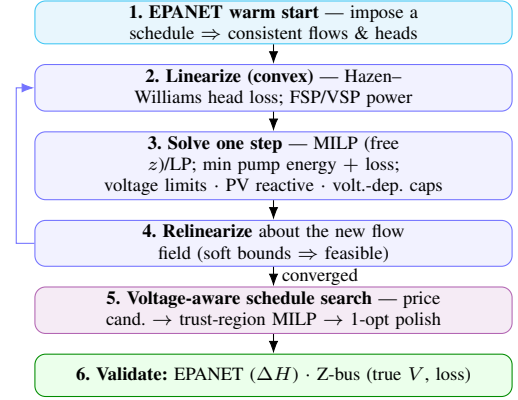


Fig. 1. The coupled C-OWPF solve. Steps 2–4 are the successive-linearization loop; step 5 wraps it in the cost- and voltage-aware schedule search.

IV. VALIDATION

The winning schedule is never trusted on its linear score alone: the water schedule is replayed in **EPANET** ($\Delta H, \Delta \text{flow}$) and the power injections in the nonlinear **Z-bus** fixed-point solver (true voltages and true $\sum |I|^2 r$ loss).

V. CODE MAP

All paths are under `python/`.

Element	Source
Water modes / driver	<code>main_owf.py</code> : <code>MODES, run_case</code>
Succ. lin. + search	<code>owf/warmstart.py</code> : <code>optimize_schedule</code>
Price / load-shift cand.	<code>owf/warmstart.py</code> : <code>price_threshold_schedules</code>
Coupled step (P2)	<code>coupled/coupled_lp.py</code> : <code>solve_coupled</code>
Trust region (z_0, K)	<code>coupled/coupled_lp.py</code> : <code>build_coupled_problem</code>
Coupled sched. search	<code>coupled/schedule.py</code> : <code>optimize_coupled_schedule</code>
LinDistFlow / branch flows	<code>pdn/lindistflow.py</code> : <code>build_lindist</code>
Nonlinear validation	<code>coupled/validation.py</code> (Z-bus) + EPANET